TileFlow: Spatial Neural Networks on AMD XDNA 2 NPU

Hardware-software co-design for close-to-metal neural network inference

Built with IRON/MLIR-AIE toolchain — Target: AMD Ryzen AI 9 HX 370 (Strix Point)

1. Introduction

Modern neural processing units (NPUs) are spatial dataflow computers: instead of a single CPU with caches and a register file, they expose a 2D array of small compute tiles, each with its own SRAM and SIMD units, connected by a programmable interconnect. This architecture is extremely efficient for *on-chip* computation — but only if the software maps the algorithm directly to the physical hardware.

TileFlow takes this literally. We design a neural network whose architecture is dictated by the physical tile layout of the AMD XDNA 2 NPU. The network has learnable parameters (a shared weight matrix) and non-linearities (ReLU), making it a valid machine-learning model — but its structure (number of parallel paths, loop depth, buffer sizes) matches the hardware exactly.

> **Key principle:** We design the network to match the hardware, not the other way around. Any architecture with learnable parameters and non-linearities can learn — so we choose the one that maximizes hardware utilization.

2. Background

This section provides the hardware and systems context that most ML engineers never need to think about — until they want to understand *why* certain hardware is fast (or slow) for their models. If you've trained models with PyTorch or JAX and think of hardware as "a GPU with some VRAM," this section fills in the gap between that abstraction and the physical reality of a spatial processor like the XDNA 2 NPU.

2.1 Glossary of acronyms

The table below defines every acronym used in this paper. We group them by domain so you can revisit this section as a reference while reading.

Acronym	Stands for	What it means
Hardware & memory		
NPU	Neural Processing Unit	A dedicated accelerator for neural-network inference (and sometimes training), built into a laptop or phone chip.
APU	Accelerated Processing Unit	AMD's name for a single die that integrates CPU + GPU + NPU.
XDNA	(AMD brand name)	AMD's NPU architecture family. XDNA 2 is the second generation, found in "Strix Point" Ryzen AI chips.
AIE	AI Engine	The individual tile processor IP inside the XDNA NPU, originally designed by Xilinx (acquired by AMD).
SRAM	Static Random-Access Memory	Fast, on-chip memory (~1–2 ns access). Each compute tile has ~64 KB of SRAM. Expensive per bit, but extremely fast because it sits right next to the compute logic.
DRAM	Dynamic Random-Access Memory	The main system memory (8–64 GB). Much slower than SRAM (~50–100 ns access) but vastly cheaper per bit.
DDR	Double Data Rate	The interface standard for DRAM modules. "DDR memory" is the system RAM your laptop uses. In this paper, "DDR" means "host-side main memory."
DMA	Direct Memory Access	A hardware mechanism that copies data between memory regions *without using the CPU*. The NPU's shim tiles contain DMA engines that move data between DDR and tile SRAM.
PCIe	Peripheral Component Interconnect Express	The high-speed bus connecting the NPU to the rest of the system.
Compute concepts		
SIMD	Single Instruction, Multiple Data	A processor executes one instruction on a *vector* of values simultaneously. For example, multiplying 32 numbers in one clock cycle instead of one at a time. This is how GPUs and NPUs achieve massive throughput.
VLIW	Very Long Instruction Word	A processor design where each instruction encodes *multiple operations* to execute in parallel (e.g., a multiply, an add, and a load all in one cycle). Each AIE tile uses a VLIW core.
MMUL	Matrix Multiply (unit)	A hardware block dedicated to multiplying small matrices (e.g., 8×8 blocks of bfloat16). This is the workhorse of each AIE tile.

FIFO	First In, First Out	A queue where data is read in the same order it was written. The NPU uses hardware FIFOs (“ObjectFIFOs”) to stream data between tiles, like Unix pipes between processes.
GEMM	General Matrix Multiply	The standard linear algebra operation C = A × B + C. Neural network layers are essentially sequences of GEMMs with non-linearities in between.
Numeric formats		
bf16 / bfloat16	Brain Floating Point, 16-bit	A 16-bit floating-point format with the same exponent range as float32 (8 bits) but less precision (7-bit mantissa vs 23-bit). Invented at Google Brain for ML training where range matters more than precision.
BFP16	Block Floating Point, 16-bit	An emulation mode on AIE2P that groups bf16 values into blocks sharing a common exponent. Enables efficient SIMD matmul in the MMUL unit with tile factor r = s = t = 8.
INT8	8-bit Integer	8-bit integer arithmetic, used for quantized inference. The NPU’s peak in INT8 mode is 50 TOPS (double the bf16 rate).
Performance metrics		
FLOPS	Floating-point Operations Per Second	The standard measure of compute throughput. One multiply-add on two numbers counts as 2 FLOPs.
GFLOPS	Giga FLOPS (10^9)	Billions of floating-point operations per second.
TFLOPS	Tera FLOPS (10^{12})	Trillions of floating-point operations per second. The NPU’s peak is 25 TFLOPS in bfloat16.
TOPS	Tera Operations Per Second	Like TFLOPS but for integer operations. Used for INT8 peak specs (50 TOPS for this NPU).
Machine learning		
MLP	Multi-Layer Perceptron	A neural network made of fully-connected (dense) layers. Each layer computes y = activation(x × W + b).
ReLU	Rectified Linear Unit	The activation function max(x, 0). Simple, cheap to compute, and widely used.
Toolchain & software		
IRON	(name, not an acronym)	AMD’s Python API for programming AIE tiles at a high level: defining kernels, ObjectFIFOs, and tile placements.

MLIR	Multi-Level Intermediate Representation	A compiler framework (from the LLVM project) that represents programs at multiple abstraction levels. MLIR-AIE is the dialect that targets AIE hardware.
XRT	Xilinx Runtime	The userspace library and kernel driver that loads bitstreams onto the NPU and manages execution.
XCLBIN	Xilinx Container for Linux Binary	The compiled binary file that contains the NPU bitstream (tile configuration, routing, kernel code).
LLVM	Low Level Virtual Machine	A widely-used compiler infrastructure. Peano/LLVM-AIE is a fork that compiles C++ to AIE tile machine code.
BD	Buffer Descriptor	A hardware structure in the DMA engine that describes one data transfer: source address, size, stride pattern. The 10-bit size field (max 1024) is a constraint discussed in Section 4.3.

2.2 How CPUs execute neural networks (and why it's slow)

When you call `y = torch.relu(x @ W)` on a CPU, here is what actually happens at the hardware level:

1. The weight matrix `W` lives in DRAM. The CPU issues a load instruction, and the data travels through the **memory hierarchy**: DRAM → L3 cache → L2 cache → L1 cache → registers.
2. The CPU's SIMD units multiply a few rows at a time. The result goes back through the cache hierarchy to DRAM.
3. For the next layer, the process repeats: read the activation from DRAM, read the next weight matrix from DRAM, compute, write back.

The fundamental problem is the **memory wall**: modern compute units can perform arithmetic far faster than memory can supply data. A typical CPU can do ~1 TFLOPS of bf16 math, but its DRAM bandwidth is ~50 GB/s. To keep the ALUs busy doing a 128×128 matmul, you need 128×128×2 = 32 KB of weight data delivered every ~1 μs — which is 32 GB/s just for one matrix. If you're running 4 layers, that's 128 GB/s, already exceeding the memory bandwidth. The CPU stalls waiting for data.

> **The memory wall in one sentence:** Arithmetic is cheap; moving data is expensive. The deeper you go in a neural network, the more time is spent shuffling data between memory levels, not doing useful math.

2.3 The spatial dataflow alternative

A **spatial architecture** takes a radically different approach. Instead of one fast processor with a deep cache hierarchy, it uses *many small processors* (tiles), each with a tiny but *extremely fast* local memory (SRAM). Data moves between tiles through dedicated hardware channels (FIFOs), not through shared caches.

Think of it like a factory assembly line versus a single master craftsman:

- **CPU (craftsman):** One highly skilled worker goes to the warehouse (DRAM) for each part, brings it to the workbench (registers), processes it, takes the result back to the warehouse, gets the next part. Most time is spent walking, not working.
- **NPU (assembly line):** Many workers sit at their stations with all their parts already on their desk (SRAM). A conveyor belt (FIFO) moves the workpiece from one station to the next. Nobody walks anywhere.

The key property is **data locality**: once data is loaded into a tile's 64 KB SRAM, it stays there for as many operations as you can do on it. There is no cache to get evicted from, no bus to contend for. If a 128×128 weight matrix (32 KB) fits in SRAM, you can multiply against it thousands of times at full speed — which is exactly what our recurrent MLP does.

2.4 Key hardware concepts used in this project

These concepts appear throughout the paper. You don't need to understand every transistor, but knowing these ideas will make the design choices clear:

Double buffering (ping-pong)

If a tile needs to *receive* new data while *computing* on data it already has, you need two buffers: one being filled by DMA while the other is being read by the compute unit. They swap roles each iteration. We use this for the activation buffers: buffer A holds the input, the tile computes into buffer B, then B becomes the input and A becomes the output.

Tiled matrix layout

A "row-major" matrix stores elements left-to-right, top-to-bottom: `[[a, b], [c, d]]` becomes `[a, b, c, d]`. The AIE matmul unit instead expects a **blocked (tiled) layout**: the matrix is divided into 8×8 sub-matrices, and each block is stored contiguously. This matches the MMUL hardware which multiplies 8×8 blocks in one operation. The `to_tiled()` function handles this conversion.

ObjectFIFOs and data movement

On a CPU, data movement is implicit — you `load` from an address and the cache hierarchy handles the rest. On the NPU, you must *explicitly* program every data transfer: "move 4 KB from DDR address X to tile (3, 2)'s input buffer." IRON's `ObjectFifo` abstraction makes this manageable: you declare a typed channel between a producer and a consumer, and the compiler generates the DMA configurations.

The invocation overhead problem

Every time the host CPU tells the NPU to run, there is a fixed overhead of ~120 μs for driver calls, instruction dispatch, and DMA setup. This is analogous to the overhead of launching a CUDA kernel on a GPU. If your actual compute takes only 1 μs (as it does for a single small matmul), you are spending 99% of the time on overhead. The solution is to do *lots of work per invocation* — hence the hardware loop that repeats thousands of matmuls before returning to the host.

2.5 How to read the rest of this paper

With this background, the rest of the paper should be accessible:

- **Section 3** describes the physical hardware: the tile grid, memory sizes, and interconnect.
- **Section 4** presents the neural network architecture and explains why each design choice follows from the hardware constraints.
- **Section 5** shows throughput and speedup results.
- **Section 6** describes the software toolchain.
- **Section 7** maps the code structure to the concepts.

3. The Hardware

The AMD XDNA 2 NPU in the Ryzen AI 9 HX 370 (codename Strix Point) is a tiled spatial-dataflow processor with the following structure:

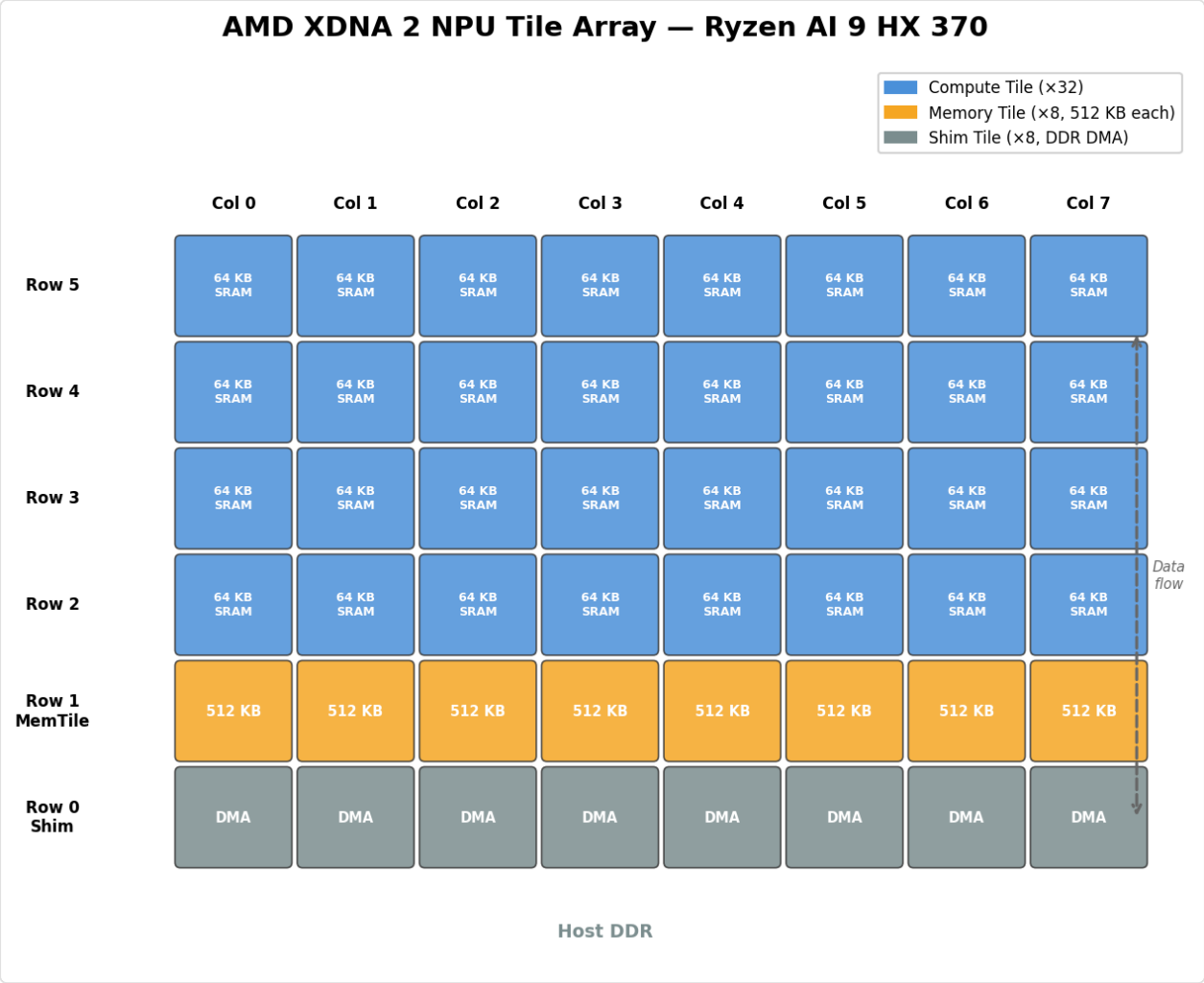

Figure 1: Physical tile array of the AMD XDNA 2 NPU. 32 compute tiles (rows 2–5) each contain ~64 KB SRAM and a bf16 MMUL unit. 8 memory tiles (row 1, 512 KB each) serve as on-chip L2 buffers and routing hubs. 8 shim tiles (row 0) provide DMA access to host DDR memory.

Property	Value
Compute tiles	32 (8 columns × 4 rows)
Memory tiles	8 (512 KB each, 4 MB total)
Per-tile SRAM	~64 KB data memory
Per-tile compute	bf16 MMUL unit, VLIW+SIMD core
Clock frequency	~1.5 GHz
Peak throughput	**25 TFLOPS** (bfloat16)
Interconnect	ObjectFIFOs (tile-to-tile double-buffered streams)
Power envelope	~6 W

3.1 Why spatial dataflow matters

On a CPU, a matrix multiply reads data from DRAM through multiple cache levels. Each layer of a neural network bounces activations through L1→L2→L3→DRAM and back. On the NPU, data stays in 64 KB tile SRAM between operations — zero cache misses, zero bus contention. This is the source of the NPU's advantage for deep, narrow computations.

> **Lesson from Phase 1:** A single large GEMM achieves only 2.49 TFLOPS on the NPU (10% of peak) because it is *memory-bandwidth limited* — data must stream from DDR. The NPU wins when data **stays on-chip**.

4. The Architecture: Recurrent MLP

We choose a recurrent MLP: a single weight matrix `W` (128×128, bfloat16) is loaded once into each tile's SRAM and applied repeatedly in a tight hardware loop:

```
x = input                       # loaded from DDR, 16×128 bf16
for i in range(num_iters):
    y = ReLU(x @ W)             # matmul + activation
    x = ReLU(y @ W)             # ping-pong: result goes back to x
output = x                      # drained to DDR
```

This is mapped to 24 compute tiles (3 rows × 8 columns), each running the same loop independently on different input samples:

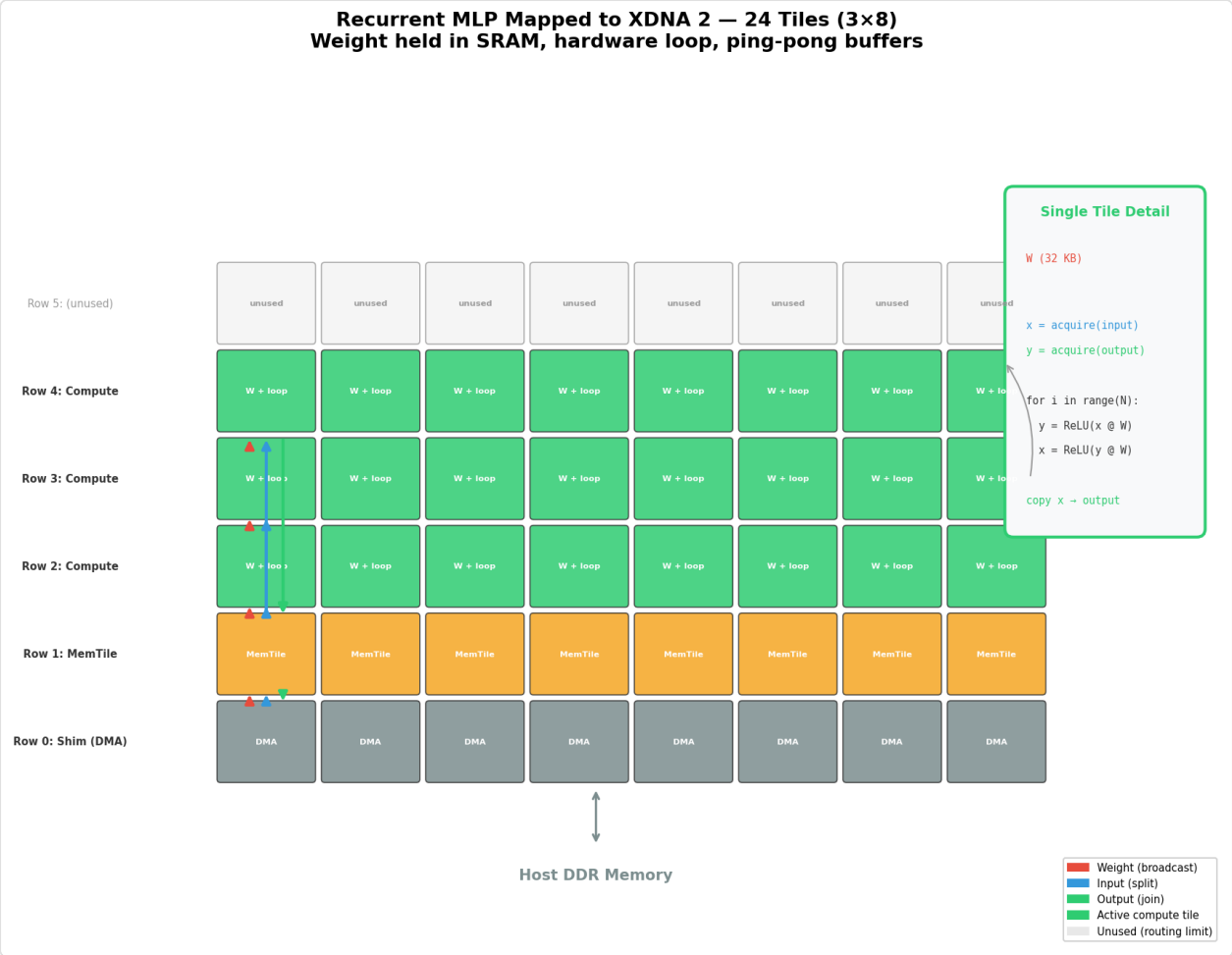

Figure 2: Recurrent MLP mapped to the XDNA 2 tile array. 24 tiles across 3 compute rows run the same hardware loop in parallel. Row 5 is unused due to MemTile routing constraints (~6 northward master ports). The detail box shows the per-tile computation: acquire buffers, loop, copy result.

4.1 Why this architecture

- **Maximizes on-chip time:** Weight is loaded once (32 KB) and reused for thousands of matmul operations. DDR I/O happens only at start and end.
- **Amortizes overhead:** Each NPU invocation has ~120 µs of driver/DMA overhead. With depth=2000, compute time (~1.3 ms) dominates overhead by 10×.
- **Fits SRAM budget:** W (32 KB) + 2 activation buffers (4 KB each) + stack (1 KB) = 41 KB, well within the 64 KB tile limit.
- **Linear tile scaling:** Each tile is independent — doubling tiles doubles throughput with no communication overhead.

4.2 Multi-row data routing

When using more than 8 tiles (i.e., more than one compute row), data must pass through the MemTiles (row 1) which act as routing hubs:

- **Weights** are *broadcast* via `forward()`: one DDR→MemTile transfer, then the MemTile fans out to all compute rows in the column.
- **Inputs** are *split* via `split()`: the host buffer is partitioned so each row gets its own batch slice.
- **Outputs** are *joined* via `join()`: per-row results are aggregated back through the MemTile to DDR.

Routing limit: Each MemTile has approximately 6 northward master ports. Our design requires 3 data streams per row (weight + input + output). At 3 compute rows = 9 streams, this fits; at 4 rows = 12 streams, the MLIR-AIE router fails. This caps us at **24 tiles** (3 rows × 8 columns).

4.3 Critical implementation constraints

Several non-obvious hardware constraints shaped the design:

- **No FIFO ops inside loops:** Placing `acquire()` / `release()` inside `range_()` (which compiles to `scf.for`) causes DMA deadlock. All FIFO operations must happen *outside* the loop.
- **DMA BD 10-bit size limit:** Shim DMA buffer-descriptor sizes are 10-bit (max 1024). For B=16, H=128, the product B×H=2048 exceeds this, so tensor access patterns must factor dimensions as [B, H] = [16, 128] instead of [B×H] = [2048].
- **Accumulating matmul:** IRON's `mm.cc` kernel computes C += A×B (not C = A×B). We must explicitly zero the output buffer before each matmul, which wastes ~12% of cycle time.

5. Results

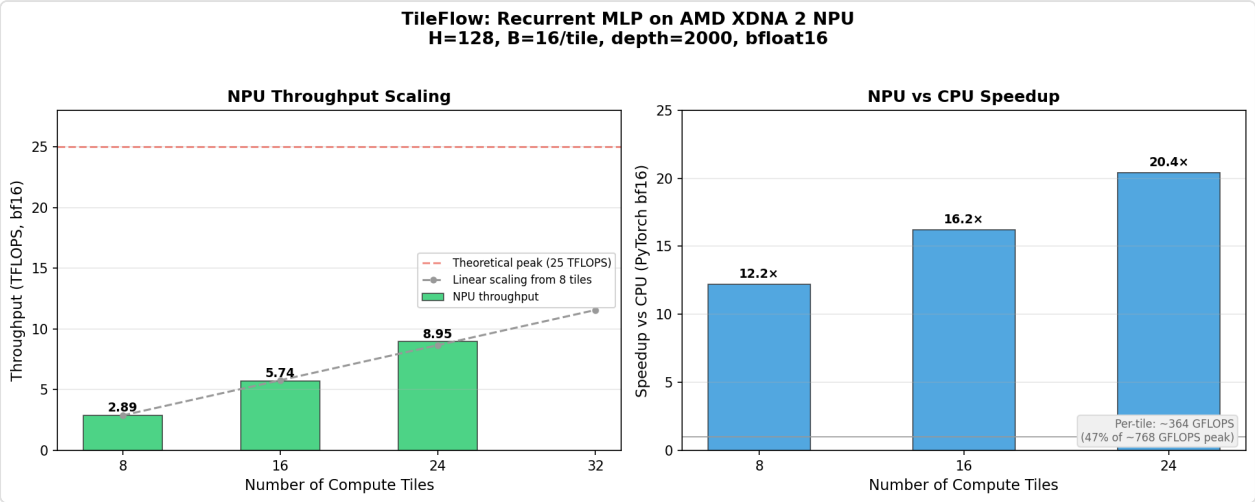

Figure 3: NPU throughput scaling (left) and speedup over CPU (right). Throughput scales near-linearly with tile count at ~360 GFLOPS/tile. At 24 tiles, we achieve 8.95 TFLOPS and 20× CPU speedup.

Tiles	Depth	NPU Latency	NPU TFLOPS	CPU GFLOPS	Speedup
8 (1 row)	1,000	1.45 ms	**2.89**	237	**12.2×**
16 (2 rows)	1,000	1.46 ms	**5.74**	354	**16.2×**
24 (3 rows)	1,000	1.46 ms	**8.63**	429	**20.1×**
24 (3 rows)	10,000	14.05 ms	**8.95**	439	**20.4×**

5.1 Scaling analysis

Per-tile throughput is remarkably consistent at ~360 GFLOPS regardless of tile count, confirming that the tiles operate independently with no contention:

```
 8 tiles × 360 GFLOPS/tile =  2.9 TFLOPS  ✓
16 tiles × 360 GFLOPS/tile =  5.7 TFLOPS  ✓  (near-linear)
24 tiles × 360 GFLOPS/tile =  8.6 TFLOPS  ✓  (near-linear)
```

5.2 Gap to theoretical peak

We achieve 8.95 of 25 TFLOPS (35.8%). The remaining gap is well-understood:

Factor	Impact	Potential fix
Per-tile utilization	360/768 = 47%	Fused C=A×B kernel

zero_bf16 overhead	~12% of step time	Fused kernel eliminates this
Array utilization	24/32 = 75%	4-row routing (needs HW/compiler support)
Combined theoretical max	~18 TFLOPS	With fused kernel + 24 tiles

6. The Toolchain

Component	Role
IRON	Python API for tile layout, ObjectFIFOs, and dataflow
MLIR-AIE	MLIR dialect → hardware bitstream compilation
Peano/LLVM-AIE	C++ compiler for per-tile kernels
XRT	Runtime for loading and executing on the NPU

6.1 Compilation pipeline

```
design.py  →  MLIR  →  aiecc  →  .xclbin (bitstream)
                          →  .bin   (instruction sequence)

mm.cc          →  mlp_mm.o    →  mlp_kernels.a
mlp_kernels.cc →  mlp_relu.o  ↗
```

7. Code Structure

The project is intentionally minimal — four Python files and one C++ file:

File	Lines	Purpose
`spatial_mlp/__init__.py`	~55	Tiling utilities (`to_tiled` , `from_tiled`)
`spatial_mlp/design.py`	~300	IRON design: tile topology, FIFOs, workers, DMA
`spatial_mlp/op.py`	~140	IRON operator: compilation artifacts, runtime buffers
`spatial_mlp/test.py`	~240	Benchmark: NPU vs CPU execution and reporting
`aie_kernels/mlp_kernels.cc`	~55	Custom AIE2P kernels: ReLU, copy (bf16, SIMD)

Each module has a single, well-defined responsibility. The design module is decomposed into small functions that each handle one aspect of the hardware mapping: validation, kernel definition, FIFO topology, worker bodies, tensor access patterns, and DMA sequences.

8. Future Work

- **Fused matmul kernel:** A C=A×B kernel (instead of C+=A×B + separate zero) would eliminate ~12% overhead and push per-tile throughput from 360 to ~700 GFLOPS.
- **INT8 mode:** The NPU's peak is 50 TOPS for int8. With H=256 (fitting larger weights), this could double throughput.
- **Training:** Research NPU backpropagation (see arXiv:2504.03083).
- **Real ML task:** Apply the architecture to a concrete sequence modeling or time-series task where deep recurrence is natural.

References

1. AMD IRON repository: github.com/amd/IRON
2. MLIR-AIE programming guide: github.com/Xilinx/mlir-aie
3. IRON tutorial (IPDPS 2025): AMD Technical Paper
4. NPU training: arXiv:2504.03083
5. Linux kernel NPU docs: kernel.org